DevOps Infrastructure – Unit 2 Notes (Part 2)

Image Creation, Tagging, and Image Inspection

---

1. Docker Image Creation in Detail

1.1 Docker Build Process

Definition

The Docker build process converts a Dockerfile into a Docker image.

Step-by-Step Process

1. User runs the build command
   docker build -t my_app:1.0 .
2. Docker sends build context to the Docker daemon
3. Docker reads the Dockerfile line by line
4. Each instruction is executed sequentially
5. For every instruction, a new layer is created
6. Layers are cached for reuse
7. Final image is created and stored locally

1.2 Layer Caching Mechanism

Docker uses caching to speed up builds.

Example:

```
FROM ubuntu
RUN apt-get update
RUN apt-get install python3
COPY app.py /app/
```

If only app.py changes:

- Only the COPY layer is rebuilt
- Previous layers are reused

Best Practice

- Place frequently changing instructions (like COPY) at the end
- Place stable instructions (like installing dependencies) at the top

---

2. Image Tagging and Versioning

2.1 What is Tagging?

Tagging assigns a name and version to an image.

Format:
repository_name:tag

Example:
my_app:1.0
my_app:latest

2.2 Why Tagging is Important

- Helps manage different versions
- Enables rollback to previous versions
- Useful in CI/CD pipelines

2.3 Common Tagging Practices

1. latest
   Default tag if none specified
2. Version-based
   v1.0, v2.1, etc.
3. Environment-based
   dev, test, prod

2.4 Tagging Commands

Build with tag:
docker build -t my_app:1.0 .

Add a new tag:
docker tag my_app:1.0 my_app:latest

---

3. Inspecting Docker Images

3.1 Viewing Image List

Command:
docker images

Shows:

- Repository name
- Tag
- Image ID
- Size

3.2 Image History

Command:
docker history my_app

Purpose

- Shows all layers of the image
- Displays commands used to build each layer
- Helps in debugging and optimization

3.3 Inspecting Image Details

Command:
docker inspect my_app

Information Provided

- Metadata

- Configuration details
- Network settings
- Storage details

3.4 Understanding Image Layers

Each image consists of multiple layers stacked together.

Important points:

- Layers are read-only
- Shared across images
- Improve efficiency

---

4. Practical Example

Build an image:
docker build -t web_app:1.0 .

Check images:
docker images

View history:
docker history web_app

Inspect image:
docker inspect web_app

---

5. Summary

- Docker build converts Dockerfile into an image
- Each instruction creates a cached layer
- Tagging helps manage versions effectively
- docker history shows layers
- docker inspect provides detailed metadata